Day 165: Calibration: When a Model Says 70%, Does It Happen 70% of the Time?

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 5 | Complete study guide

Watch: Model Calibration - Estimated Calibration Error (ECE) Explained (DataMListic) | Probability Calibration Workshop - Lesson 2 (numeristical, Brian Lucena)

Script: `day165_calibration.py` · every number in this guide is copied by program from `day165_run_output.txt`, the real output of that script.

A model can put people in the right order and still give them the wrong probabilities. Today measures that on UCI Adult with four models and fixes it. Logistic regression trained on a 50/50 sample ranked as well as the ordinary one (AUC 0.8968 against 0.8972) but predicted 2301 high earners where 1501 existed; a deep tree said 0% or 100% for almost everyone. Platt scaling and isotonic regression, fitted on held-out rows, repaired both. Three lessons came out of checking the checks: the error measure has a chance floor, pooled cross-fitted scores flatter the fixes, and a fix applied to an already calibrated model made it worse.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

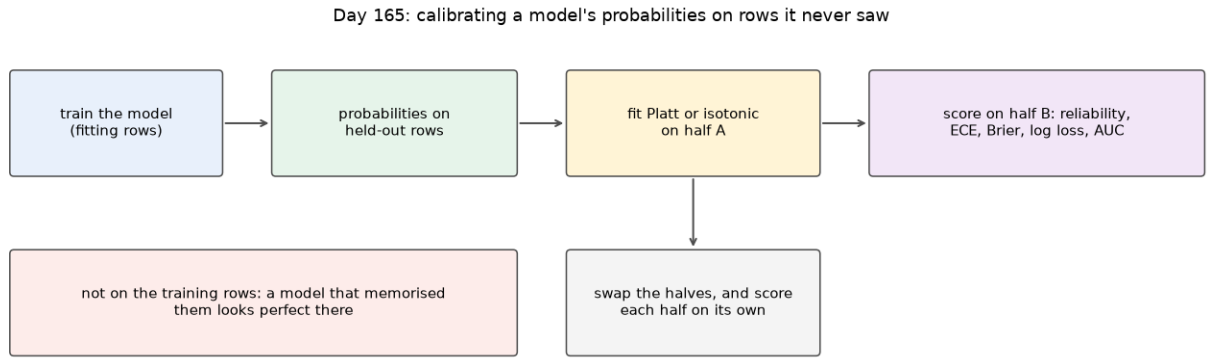
A model is **calibrated** when its probabilities mean what they say: among all the people it gives 70%, about 70% turn out positive. Calibration is separate from **ranking** (AUC): a model can order people perfectly and still say 90% where 60% is true.

A **reliability diagram** sorts the predictions into bins (0–10%, 10–20%, …) and plots each bin's average prediction against the share that really happened; a calibrated model sits on the diagonal. The **expected calibration error** (ECE) is the average distance from the diagonal, weighted by how many rows each bin holds.

REAL-WORLD SCENARIO

A bank prices loans from a default model. If the model says 2% and the true rate is 6%, every loan is priced for the wrong risk, even if the model ranks borrowers perfectly.

SYSTEM DIAGRAM



Train on the fitting rows; fit the fix on half of the held-out rows and score it on the other half; swap; never trust a fix fitted on rows the model memorised.

The data and split are Week 24's: UCI Adult's training file, 24,130 fitting rows and 6,032 validation rows; Adult's test file is left for Day 167. Four models: Day 161's **logistic regression**; the same model trained on a **balanced** sample (every high earner in the fitting rows plus an equal number of the rest, a common fix for imbalance); Day 109's **gradient boosting** (200 trees of depth 3); and Day 161's **deep tree**, grown until every

leaf is pure.

Today's part for the week's build

`day165_exercise.py` is today's toolkit (the reliability table, ECE, log loss, Platt scaling, isotonic regression and cross-fitting), each checked by hand, including the small case where plain Newton runs away and step halving lands on the worked answer. It is the fifth part of Day 167's report card.

ACTUAL OUTPUT

```
reliability of four rows: bins 0, 1 and 9 (two rows: mean 0.935, one of two happened)
  got rows [1, 1, 0, 0, 0, 0, 0, 0, 0, 2], bin 9 mean 0.935 share 0.50
ece: by hand (|0.05 - 0| + |0.15 - 1| + 2 x |0.935 - 0.5|) / 4 = (0.05 + 0.85 + 0.87) / 4 =
    0.4425; got 0.4425
log_loss of y (1, 0) at p (0.8, 0.3): by hand (-ln 0.8 - ln 0.7) / 2 = (0.2231 + 0.3567) / 2 =
    0.2899; got 0.2899
isotonic_fit of y (0, 1, 0, 1) at scores 0.1 to 0.4: 1 then 0 breaks the order, so they pool to
    0.5;
  by hand edges (0.1, 0.3, 0.4) values (0, 0.5, 1); got edges [0.1, 0.3, 0.4] values [0.0, 0.5,
      1.0]
  applied to 0.05, 0.25, 0.35, 0.5: [0.0, 0.5, 1.0, 1.0]
platt_fit on 20,000 rows built with slope 2 and intercept -1: got [1.975, -0.983]
platt_fit when every score is 0 or 1 (logit about -13.82 or +13.82), one of three wrong on each
    side:
  by hand the answer gives 1/3 and 2/3: intercept 0, slope ln 2 / 13.8155 = 0.0502
  plain Newton: [-24126.4036, 0.0]; with step halving: [0.0502, 0.0]
cross_fit with a mean-only calibrator: rows 0, 2, 4 have outcomes 0, 1, 1 (rate 2/3) and rows 1,
    3, 5 have 0, 0, 1
  (rate 1/3); each row gets the other half's rate, so by hand 0.333, 0.667, 0.333, 0.667, 0.333,
      0.667; got [0.333, 0.667, 0.333, 0.667, 0.333, 0.667]
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Measuring calibration, and the error a perfect model would still show

DEFINITION — start here, no prior knowledge assumed

Each model's validation probabilities are binned into ten equal-width bins. The ECE adds up, over the bins, the gap between the average prediction and the observed share, weighted by the bin's share of rows.

ECE is computed from a finite sample, so even a perfectly calibrated model shows some. To know that floor, outcomes are simulated from the model's own probabilities (each person is positive with exactly the probability the model gave), 200 times, and the ECE of those perfect worlds is averaged.

REAL-WORLD SCENARIO

A weather forecaster says '30% rain' on 100 days. If it rained on 29 of them the forecast is fine; on 60, it is not.

MATHEMATICS

$$ECE = \Sigma_b \, (n_b/n) \cdot |\, \text{mean prediction in bin } b - \text{share positive in bin } b \,|$$

$$Brier = \text{mean } (p - y)^2 \quad \text{log loss} = -\text{mean } [y \ln p + (1 - y) \ln(1 - p)]$$

IMPLEMENTATION

```
def reliability(y, p, bins=BINS):
    b = np.minimum((p * bins).astype(int), bins - 1)
    n = np.bincount(b, minlength=bins)
    mp = np.bincount(b, p, bins) / np.maximum(n, 1)
    my = np.bincount(b, y, bins) / np.maximum(n, 1)
    return n, np.where(n > 0, mp, np.nan), np.where(n > 0, my, np.nan)
def ece(y, p, bins=BINS):
    n, mp, my = reliability(y, p, bins); ok = n > 0
    return float(np.sum(n[ok] * np.abs(mp[ok] - my[ok])) / len(y))
```

ACTUAL OUTPUT

```
  logistic          : ECE 0.0149 (if perfectly calibrated: 0.0108 +/- 0.0025); mean probability 0.2503 against a rate of
      0.2488; AUC 0.8972, Brier 0.1102, log loss 0.3410
  balanced logistic : ECE 0.1327 (if perfectly calibrated: 0.0113 +/- 0.0028); mean probability 0.3815 against a rate of
      0.2488; AUC 0.8968, Brier 0.1353, log loss 0.4139
  boosting          : ECE 0.0096 (if perfectly calibrated: 0.0096 +/- 0.0023); mean probability 0.2478 against a rate of
      0.2488; AUC 0.9258, Brier 0.0918, log loss 0.2890
  deep tree         : ECE 0.1757 (if perfectly calibrated: 0.0025 +/- 0.0008); mean probability 0.2539 against a rate of
      0.2488; AUC 0.7609, Brier 0.1825, log loss 2.3899
  balanced logistic, bin by bin (rows: said -> happened):
    1940: 0.04->0.01  656: 0.15->0.05  413: 0.25->0.09  393: 0.35->0.15  407: 0.45->0.25  373: 0.55->0.32  393: 0.65->0.42
       409: 0.75->0.50  524: 0.85->0.64  524: 0.95->0.84
  deep tree, bin by bin (rows: said -> happened):
    4338: 0.00->0.12  11: 0.13->0.09  52: 0.25->0.29  52: 0.33->0.37  8: 0.41->0.62  123: 0.50->0.41  26: 0.66->0.46  11:
       0.75->0.27  6: 0.80->0.33  1405: 1.00->0.63
  the ECE depends on the bins: logistic with 5, 10, 20, 50 bins: 0.0128, 0.0149, 0.0172, 0.0298
Actual output from this lesson's run.
```

What it means

Logistic regression and boosting were close to calibrated: ECE 0.0149 against a chance floor of 0.0108, and 0.0096 against 0.0096. The balanced model ranked as well as the ordinary one but its ECE was 0.1327: it was trained in a world where half the people are high earners, so it predicts a mean of 0.3815 where the rate is 0.2488. In its top bin it said 0.95 and 0.84 happened.

The deep tree is the extreme case: it put 4338 people at 0% (of whom 12% were high earners) and 1405 at 100% (63%). Its ECE was 0.1757 and its log loss 2.3899, because a confident wrong answer costs a great deal. The ECE also depends on the bins: for logistic regression 0.0128 with 5 bins and 0.0298 with 50, since more bins mean fewer rows per bin and more noise.

WHY CHOOSE THIS?

A reliability diagram shows where a model is wrong (top, bottom, middle); the ECE puts that in one number.

WHY NOT THIS? (TRADEOFFS)

ECE is never 0 even for a perfect model, depends on the bin count, and says nothing about ranking.

2. Two fixes, fitted on held-out rows

DEFINITION — start here, no prior knowledge assumed

Platt scaling fits a logistic regression with one input, the model's own log-odds: the fixed probability is sigmoid(a · logit(p) + b). It can only stretch and shift the S-curve, so it needs few rows.

Isotonic regression fits any non-decreasing step function from the score to the probability, by the pool-adjacent-violators algorithm: walk through the scores in order, and whenever a group's observed share is lower than the group before it, merge the two into their average. It can fix any shape, but it needs more rows.

Both must be fitted on rows the model was not trained on. The validation rows are split into two random halves; each fix is fitted on one half and applied to the other, then the halves swap (**cross-fitting**).

REAL-WORLD SCENARIO

A thermometer reads 3 degrees high on cold days and 1 degree high on warm ones. A correction table, made against a reference thermometer, fixes every reading without changing their order.

MATHEMATICS

Platt: $q = 1 / (1 + e^{-(a \cdot logit(p) + b)})$, a and b minimise the log loss; $a > 0$ keeps the order

isotonic: f minimises $\Sigma\,(y_i - f(p_i))^2$ with f non-decreasing

IMPLEMENTATION

```
def platt_fit(p, y, iters=100, damped=True):
    A = np.c_[logit(p), np.ones(len(p))]; w = np.array([1.0, 0.0])
    for _ in range(iters):
        q = 1 / (1 + np.exp(-np.clip(A @ w, -500, 500)))
        g = A.T @ (q - y)
        H = (A * (q * (1 - q))[:, None]).T @ A + 1e-9 * np.eye(2)
        step = np.linalg.solve(H, g); t = 1.0
        if damped:
            while platt_loss(A, y, w - t * step) > platt_loss(A, y, w) and t > 1e-10:
                t /= 2
        w = w - t * step
        if np.abs(t * step).max() < 1e-10:
            break
    return w
```

ACTUAL OUTPUT

```
cross-fitted: each calibrator is fitted on one half of the validation rows (2959 or 3073) and applied to the other
  first draft (plain Newton) on the deep tree: slope 1.53e+13, intercept -1.72e+13; with step halving: slope 0.0945,
      intercept -0.7815
  the halves' own rates differ by chance: 0.2410 and 0.2564 above $50K
  logistic:
    raw      : ECE on half A 0.0221, half B 0.0189 (chance floor 0.0148; both halves pooled: 0.0149); Brier 0.1102, log loss
        0.3410, mean 0.2503
    Platt    : ECE on half A 0.0241, half B 0.0268 (chance floor 0.0147; both halves pooled: 0.0167); Brier 0.1106, log loss
        0.3421, mean 0.2483
    isotonic : ECE on half A 0.0291, half B 0.0295 (chance floor 0.0141; both halves pooled: 0.0107); Brier 0.1113, log loss
        0.3472, mean 0.2483
  balanced logistic:
    raw      : ECE on half A 0.1393, half B 0.1263 (chance floor 0.0163; both halves pooled: 0.1327); Brier 0.1353, log loss
        0.4139, mean 0.3815
    Platt    : ECE on half A 0.0230, half B 0.0269 (chance floor 0.0147; both halves pooled: 0.0142); Brier 0.1111, log loss
        0.3428, mean 0.2483
    isotonic : ECE on half A 0.0324, half B 0.0287 (chance floor 0.0126; both halves pooled: 0.0134); Brier 0.1124, log loss
        0.3528, mean 0.2481
  boosting:
    raw      : ECE on half A 0.0193, half B 0.0142 (chance floor 0.0134; both halves pooled: 0.0096); Brier 0.0918, log loss
        0.2890, mean 0.2478
    Platt    : ECE on half A 0.0219, half B 0.0213 (chance floor 0.0137; both halves pooled: 0.0088); Brier 0.0922, log loss
        0.2899, mean 0.2482
    isotonic : ECE on half A 0.0278, half B 0.0263 (chance floor 0.0130; both halves pooled: 0.0091); Brier 0.0938, log loss
        0.2996, mean 0.2498
  deep tree:
    raw      : ECE on half A 0.1723, half B 0.1794 (chance floor 0.0037; both halves pooled: 0.1757); Brier 0.1825, log loss
        2.3899, mean 0.2539
```

```
    Platt    : ECE on half A 0.0182, half B 0.0166 (chance floor 0.0089; both halves pooled: 0.0048); Brier 0.1402, log loss
        0.4461, mean 0.2485
    isotonic : ECE on half A 0.0164, half B 0.0171 (chance floor 0.0095; both halves pooled: 0.0025); Brier 0.1400, log loss
        0.4456, mean 0.2485
  boosting after Platt: mean probability 0.2573 on half A (rate 0.2410), 0.2393 on half B (rate 0.2564): each half gets
      close to the other half's rate
  AUC within half B, where one map is used for every row (logistic): raw 0.8896, Platt 0.8896, isotonic 0.8888
Actual output from this lesson's run.
```

What it means

The first draft of Platt scaling used plain Newton steps. On the deep tree, whose scores are almost all 0 or 1 (log-odds of about ±13.8), it ran away to a slope of 1.53e+13, which turns every probability straight back into 0 or 1. Step halving (never take a step that raises the loss) fixed it: slope 0.0945. On six rows in the exercise, plain Newton even reversed the order (slope −24126).

For the two badly calibrated models, both fixes worked. The balanced model's ECE per half fell from about 0.13 to 0.0230 and 0.0269 (Platt); the deep tree's from about 0.17 to 0.0182 and 0.0166, with its log loss from 2.3899 to 0.4456 (isotonic).

For the two that were already calibrated, both fixes made things worse. Logistic regression went from 0.0221 and 0.0189 to 0.0241 and 0.0268 with Platt, and Brier and log loss rose slightly. The run shows why: the halves' own rates differ by chance (0.2410 and 0.2564), and each fix copies the rate of the half it was fitted on. After Platt, boosting averaged 0.2573 on half A, close to half B's rate, and 0.2393 on half B.

Pooling the two halves hides this, because the two errors point in opposite directions and cancel within the bins. The deep tree after isotonic scored 0.0025 pooled, but 0.0164 and 0.0171 on each half alone. Every ECE in this guide after a fix is per half for that reason. Ranking: within one half, one Platt map left the AUC unchanged (0.8896); isotonic's ties lowered it slightly (0.8888).

WHY CHOOSE THIS?

Repairs badly calibrated models cheaply, without retraining, and (Platt) without changing the ranking.

WHY NOT THIS? (TRADEOFFS)

A fix carries the noise of the rows it was fitted on; on a model that is already calibrated it only adds that noise.

3. The trap: fitting the fix on the training rows

DEFINITION — start here, no prior knowledge assumed

It is tempting to fit the calibrator on the same rows the model was trained on: there are four times as many of them. Whether that works depends on whether the model memorised those rows.

REAL-WORLD SCENARIO

A student marks their own homework with the answer key they copied from: every answer looks right.

ACTUAL OUTPUT

```
  deep tree         : training-row log loss 0.0384 against validation 2.3899; validation ECE 0.1757 raw, after a fix fitted
      on the training rows: Platt 0.1756, isotonic 0.1757
  boosting          : training-row log loss 0.2756 against validation 0.2890; validation ECE 0.0096 raw, after a fix fitted
      on the training rows: Platt 0.0129, isotonic 0.0107
  balanced logistic : training-row log loss 0.4076 against validation 0.4139; validation ECE 0.1327 raw, after a fix fitted
      on the training rows: Platt 0.0128, isotonic 0.0076
Actual output from this lesson's run.
```

What it means

The deep tree memorised its training rows (training log loss 0.0384, validation 2.3899). On those rows its 0% and 100% look correct, so a calibrator fitted there learns to change nothing: validation ECE stayed at 0.1756 (Platt) and 0.1757 (isotonic).

The balanced logistic regression did not memorise (training log loss 0.4076, validation 0.4139), and a fix fitted on its 24,130 training rows worked well: validation ECE 0.0128 (Platt) and 0.0076 (isotonic), on all 6,032 rows with one map. The rule: fit the fix on rows the model has not seen, unless training and validation loss are known to agree.

WHY CHOOSE THIS?

Held-out rows give an honest calibrator for any model.

WHY NOT THIS? (TRADEOFFS)

They cost data; training rows give more, but only for models that have not memorised them.

4. How many rows does each fix need?

DEFINITION — start here, no prior knowledge assumed

Each fix is fitted on 100, 300, 1,000 or all 2,959 rows of half A (50 random draws for the smaller sizes) and scored on half B.

REAL-WORLD SCENARIO

A small clinic wants to recalibrate a published risk score on its own patients. It has 200 records: which fix?

ACTUAL OUTPUT

```
  balanced logistic: ECE on half B (3073 rows) raw 0.1263; perfectly calibrated would give about 0.0163
    Platt    : mean ECE on half B with 100, 300, 1000, 2959 calibration rows: 0.0484, 0.0338, 0.0277, 0.0269
    isotonic : mean ECE on half B with 100, 300, 1000, 2959 calibration rows: 0.0648, 0.0456, 0.0335, 0.0287
  boosting: ECE on half B (3073 rows) raw 0.0142; perfectly calibrated would give about 0.0139
    Platt    : mean ECE on half B with 100, 300, 1000, 2959 calibration rows: 0.0416, 0.0294, 0.0229, 0.0213
    isotonic : mean ECE on half B with 100, 300, 1000, 2959 calibration rows: 0.0537, 0.0376, 0.0273, 0.0263
Actual output from this lesson's run.
```

What it means

Platt scaling needed fewer rows. On the balanced model, with 100 rows Platt reached 0.0484 and isotonic 0.0648; with all 2,959 the two were close (0.0269, 0.0287). Both stayed above half B's chance floor; a likely reason, not tested separately here, is the difference between the halves' rates (section 2).

On boosting, which needed no fix (raw 0.0142 on half B), every fix at every size was worse than leaving it alone.

WHY CHOOSE THIS?

Platt: two numbers to fit, stable with a few hundred rows.

WHY NOT THIS? (TRADEOFFS)

Platt assumes an S-shaped error; isotonic fixes any shape but needs thousands of rows and produces ties.

5. Why it matters: totals, decisions, and groups

DEFINITION — start here, no prior knowledge assumed

Calibration matters whenever probabilities are added up, compared with a cost, or compared across groups. Three checks: the predicted number of high earners against the true number; who would be acted on at a 50% threshold; and calibration within women and within men separately.

REAL-WORLD SCENARIO

A charity plans next year's budget from the sum of each donor's predicted chance of giving. A model that over-predicts by half plans for money that will not arrive.

ACTUAL OUTPUT

```
  1501 of the 6032 validation people earn above $50K
  logistic (raw): predicted total 1510; above 50%: 1241 people, of whom 0.718 really are; accuracy at 0.5: 0.8408
  balanced logistic (raw): predicted total 2301; above 50%: 2223 people, of whom 0.567 really are; accuracy at 0.5: 0.8007
  balanced logistic (Platt): predicted total 1498; above 50%: 1252 people, of whom 0.709 really are; accuracy at 0.5: 0.8380
  logistic Female (1925): mean probability 0.1173 against a rate of 0.1174; ECE 0.0208 (chance floor 0.0135)
  logistic Male   (4107): mean probability 0.3126 against a rate of 0.3104; ECE 0.0178 (chance floor 0.0139)
  boosting Female (1925): mean probability 0.1167 against a rate of 0.1174; ECE 0.0128 (chance floor 0.0119)
  boosting Male   (4107): mean probability 0.3093 against a rate of 0.3104; ECE 0.0101 (chance floor 0.0132)
Actual output from this lesson's run.
```

What it means

The balanced model predicted 2301 high earners where there were 1501, and flagged 2223 people above 50%, of whom only 0.567 were high earners. After Platt it predicted 1498, flagged 1252, 0.709 correct, and its accuracy at 0.5 rose from 0.8007 to 0.8380, close to the ordinary model's 0.8408.

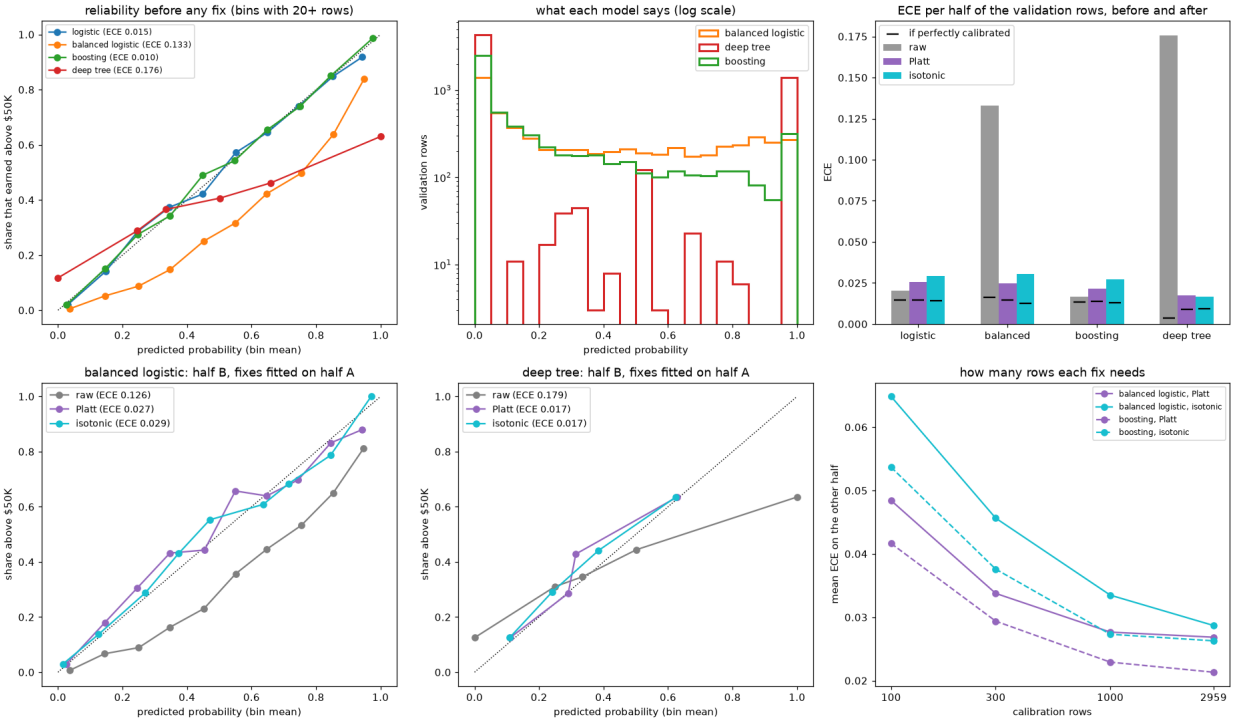
Within each sex, logistic regression's average probability matched the group's rate (women 0.1173 against 0.1174); its ECE was 0.0208 for women against a chance floor of 0.0135, a little above it, and boosting's 0.0128 against 0.0119. Calibration within groups is one of the fairness criteria Day 166 tests; when groups' base rates differ, as here, it can conflict with the others.

WHY CHOOSE THIS?

Calibrated probabilities can be summed, compared with costs, and trusted per group.

WHY NOT THIS? (TRADEOFFS)

Calibration overall does not guarantee calibration within every group; it must be checked group by group.



Top: reliability before any fix; how each model's probabilities are spread; ECE per half before and after the fixes, with each version's chance floor. Bottom: the balanced model and the deep tree on half B, fixes fitted on half A; how the mean ECE on half B falls as the calibration rows grow.

Interview preparation — technical questions

Q: A model has a high AUC. Can its probabilities still be wrong?

A: Yes: the balanced model had AUC 0.8968, as good as the ordinary one, but predicted 2,301 high earners where there were 1,501.

Q: How do you check calibration?

A: A reliability diagram and the ECE on held-out rows, compared with the ECE a perfectly calibrated model would show by chance (0.0108 here for logistic regression).

Q: Platt or isotonic?

A: Platt with few rows (ECE 0.0484 against 0.0648 with 100 rows); isotonic only with thousands, when the error is not S-shaped.

Q: Why not fit the calibrator on the training rows?

A: A model that memorised them looks perfect there: the deep tree's ECE stayed at 0.1757 after such a fix.

Q: You cross-fit a calibrator and pool the halves. What can go wrong?

A: Opposite errors cancel: the deep tree scored 0.0025 pooled but about 0.017 per half.

Interview preparation — theoretical questions

Q: Why does training on a balanced sample break calibration?

A: The model learns the 50/50 base rate of its training data; its probabilities are right for that world and too high for the real one.

Q: Does Platt scaling change the AUC?

A: Not with one map and a positive slope: AUC 0.8896 before and after on half B.

Q: Why did recalibrating an already calibrated model hurt?

A: The fix copies the base rate and noise of the rows it was fitted on; the halves' rates differed by chance (0.2410 and 0.2564).

Q: Is ECE a reliable number?

A: It depends on the bins (0.0128 with 5 bins, 0.0298 with 50) and has a chance floor; compare models on the same rows and bins.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Ranking and probability are different promises. Check calibration on held-out rows against the error chance alone would give; fix it with Platt (few rows) or isotonic (many) fitted on rows the model never saw; score each half on its own; and leave a calibrated model alone.

- Balanced logistic: AUC 0.8968, ECE 0.1327; after Platt per half 0.0230, 0.0269.
- Deep tree: log loss 2.3899 to 0.4456 after isotonic.
- Pooled halves flatter: 0.0025 pooled against 0.0164 and 0.0171 per half.
- Fix on memorised training rows: ECE 0.1757, unchanged.

Suggested watch order, if starting from zero

- 1. Model Calibration - Estimated Calibration Error (ECE) Explained (DataMListic) — ECE and the reliability curve, from the start.
- 2. Probability Calibration Workshop - Lesson 2 (numeristical, Brian Lucena) — reliability diagrams, Platt scaling and isotonic regression on real data.

Day 166 preview

- **Fairness.** The same Adult models checked across groups: demographic parity, equalized odds and calibration within groups, why they cannot all hold at once, and what a separate threshold per group changes.